Práctica 3: Vectorización aplicada a un problema real: procesado de imagen
30237 Multiprocesadores - Grado Ingeniería Informática
Esp. en Ingeniería de Computadores

Jesús Alastruey Benedé y Víctor Viñals Yúfera
Área Arquitectura y Tecnología de Computadores
Departamento de Informática e Ingeniería de Sistemas
Escuela de Ingeniería y Arquitectura
Universidad de Zaragoza

Marzo-2026

Figure 1: Annapurna I (8.091 m) desde el campo base (Nepal)

Departamento de Informática e Ingeniería de Sistemas
Universidad Zaragoza

Escuela de Ingeniería y Arquitectura
Universidad Zaragoza

Resumen

El objetivo de esta práctica es aplicar conocimientos adquiridos en las dos sesiones anteriores a una aplicación de procesado de imagen. Vamos a abordar la conversión de una imagen de formato RGB a YCbCr.

Ficheros y directorios de trabajo

- `jpeg_handler.c`: funciones para lectura y escritura de imágenes en formato JPEG.
- `RGB2YCbCr.c`: funciones para convertir una imagen de formato RGB a YCbCr.
- `misc.c`: funciones diversas (medida de tiempos, comparación de imágenes, lectura y escritura de imágenes en formato PPM, PGM …).
- `dummy.c`: su objetivo es forzar que el compilador genere código que ejecute el bucle de trabajo un número especificado de repeticiones.
- `test_RGB2YCbCr.c`: programa que permite ejecutar y verificar las distintas funciones en `RGB2YCbCr.c`.
- `Makefile` y `Makefile.icc`: para compilar los ficheros fuente.
- `images/`: contiene una fotografía en formato JPEG, que es la imagen a procesar. Este directorio almacenará las imágenes generadas así como la referencia para la verificación de resultados.

Consideraciones previas

Requerimientos hardware y software:

- CPU con soporte de la extensión vectorial AVX
- SO Linux

Los equipos de los laboratorios del DIIS cumplen los requisitos indicados. Puede trabajarse en dichos equipos de forma presencial y en los del L1.02 también de forma remota. En el repositorio de prácticas de la asignatura hay un documento que explica cómo descubrir equipos disponibles o encenderlos de forma remota.

Parte 0. Biblioteca de funciones JPEG (libjpeg)

En el fichero `jpeg_handler.c` se encuentran las funciones que vamos a utilizar para leer y escribir imágenes en formato JPEG:

```
int  read_jpeg_file(char *filename, image_t *image);
/* char *filename: nombre del fichero que contiene la imagen a leer */
/* puntero a una estructura con información de la imagen leída
   (altura, anchura, número de canales) y el valor de los píxeles de la misma
   (3 bytes por pixel para imágenes RGB,
    1 byte por pixel para imágenes en escala de grises)
   Imagen almacenada por filas desde la esquina superior izquierda
   En caso de imagen RGB, los valores RGB se almacenan entrelazados:
       RGB_pixel0  RGB_pixel1  RGB_pixel2 ...
 */

int write_jpeg_file(char *filename, image_t *image);
/* char *filename: nombre del fichero que contiene la imagen a escribir */
/* puntero a una estructura con información sobre la imagen a escribir */
```

Ambas funciones hacen uso de la biblioteca `libjpeg`, software que implementa compresión y descompresión de imágenes en formato JPEG. En este enlace puedes obtener información sobre `libjpeg`: http://www.ijg.org/

Usaremos la versión de la biblioteca `libjpeg` instalada en el sistema:

```
$ rpm -q libjpeg-turbo
```

Parte 1. Conversión de formato RGB a YCbCr

En esta parte vamos a trabajar con el fichero `RGB2YCbCr.c`.

1. La función `RGB2YCbCr_roundf0()` convierte una imagen en formato RGB a YCbCr (YUV444). Los componentes YCbCr de cada píxel de la imagen de salida se calculan de la siguiente forma:

```
Y  = roundf(0.299*R   +  0.587*G   +  0.114*B)
Cb = roundf(128 - 0.168736*R - 0.331264*G +  0.500*B)
Cr = roundf(128 -   0.5*R    - 0.418688*G - 0.081312*B)
```

Siendo (R,G,B) los componentes RGB de cada píxel de la imagen sobre la que se aplica la conversión.

En el siguiente enlace se describen la conversión de formato RGB a YCbCr:

https://en.wikipedia.org/wiki/YCbCr#JPEG_conversion

Analiza el código que realiza la conversión. Ayuda: los valores RGB de los píxeles están almacenados en forma de vector lineal con la siguiente disposición:

```
R0 G0 B0 R1 G1 B1 ... Rn-1 Gn-1 Bn-1
```

2. Compila el programa `test_RGB2YCbCr.c`:

   ```
   $ make test_RGB2YCbCr.c
   ```

   Ejecuta la función `RGB2YCbCr_roundf0()` desde el programa `test_RGB2YCbCr`:

   ```
   $ ./test_RGB2YCbCr -c0 -r
   ```

   Verifica que se han generado dos imágenes en formato YCbCr a partir de la imagen entrada: una en formato JPEG y otra en formato PPM. Esta última la usaremos como referencia para validar los resultados de otros códigos.

3. Observar el informe del compilador. ¿Ha vectorizado el bucle interno en `RGB2YCbCr_roundf0()`?
   Analiza el fichero que contiene el ensamblador y busca las instrucciones vectoriales (si existen) correspondientes al bucle en `RGB2YCbCr_roundf0()`.

4. La función `RGB2YCbCr_roundf1()` incluye cambios dirigidos a la vectorización de su bucle de cálculo. Las técnicas aplicadas fueron vistas en la práctica anterior.
   Observar el informe del compilador. ¿Ha vectorizado el bucle interno en `RGB2YCbCr_roundf1()`?
   Analiza el fichero ensamblador y confirma tu respuesta anterior.

   Ejecuta la función `RGB2YCbCr_roundf1()` desde el programa `test_RGB2YCbCr`:

   ```
   $ ./test_RGB2YCbCr -c1
   ```

5. La función `RGB2YCbCr_cast0()` sustituye la función de redondeo `roundf()` por un cast. Observar el informe del compilador. ¿Ha vectorizado el bucle interno en `RGB2YCbCr_cast0()`?
   Analiza el fichero que contiene el ensamblador y busca el código asociado a dicho bucle.

   Ejecuta la función `RGB2YCbCr_cast0()` desde el programa `test_RGB2YCbCr`:

   ```
   $ ./test_RGB2YCbCr -c2
   ```

6. La función `RGB2YCbCr_cast1()` utiliza en la conversión constantes de tipo `float` en lugar de `double`. Para ello se añade el sufijo `f`, por ejemplo: `0.5f`. Observar el informe del compilador. ¿Ha vectorizado el bucle interno en `RGB2YCbCr_cast1()`?
   Analiza el fichero que contiene el ensamblador y busca el código asociado a dicho bucle.

   Ejecuta la función `RGB2YCbCr_cast1()` desde el programa `test_RGB2YCbCr`:

   ```
   $ ./test_RGB2YCbCr -c3
   ```

   Calcula su aceleración respecto `RGB2YCbCr_cast0()`.

7. La función `RGB2YCbCr_cast2()` es una variante de la anterior a la que se ha eliminado la suma del valor 0.5. Ejecuta la función `RGB2YCbCr_cast2()` desde el programa `test_RGB2YCbCr`:

   ```
   $ ./test_RGB2YCbCr -c4
   ```

   Observa que la función que compara la imagen de salida con la de referencia detecta diferencias en casi la mitad de los píxeles. Compara de forma visual las diferencias de la imagen generada respecto a la de referencia.

   Calcula la aceleración respecto `RGB2YCbCr_cast0()`.

8. **OPTATIVO**. Elimina los `pragmas` de la función `RGB2YCbCr_cast1()`. Evalúa sus prestaciones.

9. **OPTATIVO**. Elimina los `restricts` de la función `RGB2YCbCr_cast1()`. Evalúa sus prestaciones.

10. La función `RGB2YCbCr_cast_esc()` es una variante escalar de `RGB2YCbCr_cast1()`. En su declaración está la directiva que impide la vectorización. Ejecuta la función `RGB2YCbCr_cast_esc()`:

    ```
    $ ./test_RGB2YCbCr -c5
    ```

    Calcula la aceleración de `RGB2YCbCr_cast1()` respecto `RGB2YCbCr_cast_esc()`.

Parte 2. Transformación en la disposición de datos

En esta parte vamos a modificar la disposición (*layout*) de los datos de la imagen para mejorar la eficiencia de los cálculos. En el caso de una imagen RGB, podemos cambiar de una organización de datos en formato vector de estructuras (*Array of Structures*, AoS):

```
R0 G0 B0 R1 G1 B1 ... Rn-1 Gn-1 Bn-1
```

a otra en formato estructura de vectores (*Structure of Arrays*, SoA):

```
R0 R1 ... Rn-1 G0 G1 ... Gn-1 B0 B1 ... Bn-1
```

Hay otras disposiciones posibles, como por ejemplo, una híbrida:

```
R0 R1 ... Rk-1 G0 G1 ... Gk-1 B0 B1 ... Bk-1 Rk Rk+1 ...
```

En el siguiente enlace se describe una transformación de AoS a SoA:

https://www.intel.com/content/www/us/en/developer/articles/technical/memory-layout-transformations.html

1. Completa la función `RGB2YCbCr_SOA0()` para que transforme la disposición de los datos de la imagen RGB de AoS a SoA antes de realizar los cálculos para la conversión del formato. Efectúa la conversión de formato RGB a YCbCr como en la función `RGB2YCbCr_cast1()`.

2. Verifica que la conversión funciona correctamente. Para ello, recompila y ejecuta el programa `test_RGB2YCbCr.c`:

   ```
   $ make test_RGB2YCbCr
   $ ./test_RGB2YCbCr -c6
   ```

   Comprueba que la imagen YCbCr generada se corresponde con la imagen de referencia.
   Calcula la aceleración respecto `RGB2YCbCr_cast1()`.

3. Analiza el fichero que contiene el ensamblador y busca las instrucciones vectoriales correspondientes al bucle interno en `RGB2YCbCr_SOA0()`.

4. Implementa la función `RGB2YCbCr_SOA1()` como una variante de `RGB2YCbCr_SOA0()` en la que se cuenta el tiempo de la transformacion de datos. Evalúa sus prestaciones:

   ```
   $ ../test_RGB2YCbCr -c7
   ```

   Calcula la aceleración respecto `RGB2YCbCr_cast0()`.

5. Escribe una función `RGB2YCbCr_block()` que entrelace la transformación de los datos con los cálculos a realizar. De esta forma, las variables auxiliares almacenarán en formato SoA **parte** de los valores RGB (en concreto, `BLOCK` píxeles) en lugar de **todos** los valores RGB de la imagen.

6. Verifica que la función de conversión funciona correctamente.

   ```
   $ make test_RGB2YCbCr
   $ ./test_RGB2YCbCr -c8
   ```

   Calcula la aceleración respecto `RGB2YCbCr_cast1()`.

7. **OPTATIVO**. Trata de reducir el tiempo de ejecución de `RGB2YCbCr_block()` cambiando el valor de `BLOCK`.

8. Compara el tiempo de ejecución de las distintas funciones:

   ```
   $ ./test_RGB2YCbCr -c9
   ```

   Ten presente que el tiempo de ejecución de `RGB2YCbCr_SOA0()` no incluye la transformación de datos, mientras que el tiempo de ejecución de `RGB2YCbCr_block()` **sí** lo hace.

9. Elimina el flag `-ffast-math` en el fichero `Makefile`. Recompila y evalúa las distintas funciones:

   ```
   $ make clean
   $ make test_rgb2ycbcr
   $ ./test_rgb2ycbcr -c9
   ```

Optativo

Repetir los puntos anteriores con el compilador icx.

Bibliografía relacionada

- Understanding YUV data formats.
  https://www.flir.com/support-center/iis/machine-vision/knowledge-base/understanding-yuv-data-formats/